

THE UNIVERSITY OF
SYDNEY

Literature Review

**Building a Privacy-Preserving Federated Learning
System with Secure Aggregation: Application to Wind
Farm SCADA Telemetry**

Author

Suryansh Shekhawat

Supervisor

Dr. Ali Shakiba

Bachelor of Engineering (Honours)

University of Sydney

September 1, 2026

Contents

Abbreviations	2
Notation	3
1 Introduction	4
2 Preliminaries	6
2.1 Federated Learning	6
2.2 Secure Multiparty Computation and Secure Aggregation	9
3 Privacy in Federated Learning	13
3.1 Threat Landscape	13
3.2 Cryptographic Defences	17
3.3 Practical Deployments and Hybrid Approaches	20
4 FL for Industrial Time-Series	22
4.1 Characteristics of Industrial Sensor Data	22
4.2 Wind Turbine SCADA: Structure and the CARE Dataset	24
4.3 Federated Learning Applied to Industrial and Wind Energy Domains . . .	25
4.4 System-Level Challenges of the Cross-Silo Regime	27
5 Gap Analysis	29
5.1 Identified Gaps	29
5.2 Research Questions	30
6 Conclusion	31
6.1 Trends Observed	31
6.2 Conclusions	31

Abbreviations

CARE	Coverage, Accuracy, Reliability and Earliness
CSV	Comma-Separated Values
DH	Diffie–Hellman (key exchange)
DP	Differential Privacy
DP-SGD	Differentially Private Stochastic Gradient Descent
FedAvg	Federated Averaging
FedMA	Federated Matched Averaging
FedProx	Federated Proximal (optimisation)
FL	Federated Learning
GDPR	General Data Protection Regulation
HE	Homomorphic Encryption
IID	Independent and Identically Distributed
IoT	Internet of Things
MSE	Mean Squared Error
NaN	Not a Number
RL	Reinforcement Learning
RMSE	Root Mean Squared Error
RNN	Recurrent Neural Network
SCADA	Supervisory Control and Data Acquisition
SCAFFOLD	Stochastic Controlled Averaging for Federated Learning
SecAgg	Secure Aggregation
SGD	Stochastic Gradient Descent
SGX	Software Guard Extensions (Intel)
SMC	Secure Multiparty Computation
TEE	Trusted Execution Environment
WT	Wind Turbine

Notation

K	Number of federated clients participating in training
k	Index of an individual client, $k \in \{1, \dots, K\}$
$\mathcal{D}_k$	Local dataset held by client k
n_k	Number of samples in $\mathcal{D}_k$
n	Total number of samples across all clients, $n = \sum_k n_k$
t	Communication round index
E	Number of local training epochs per round
w_t	Global model parameters at round t
w_k^{t+1}	Local model parameters produced by client k at round $t + 1$
Δw_k	Gradient update transmitted by client k to the aggregator
$\mathcal{L}$	Loss function
μ	FedProx proximal regularisation coefficient
w_u	Local update of client u in the SecAgg protocol
$\hat{w}_u$	Masked update transmitted by client u
$s_{u,v}$	Pairwise pseudorandom mask shared between clients u and v
b_u	Private self-mask of client u
$\hat{w}$	Recovered aggregate, $\hat{w} = \sum_k w_k$
$\mathcal{M}$	Randomised mechanism (differential privacy)
D, D'	Adjacent datasets differing in one record
ε	Differential privacy budget (smaller = stronger privacy)
δ	Failure probability parameter in (ε, δ) -DP
C	Per-sample gradient clipping bound in DP-SGD
σ	Noise multiplier for the Gaussian mechanism
t_c	Criticality threshold in the CARE dataset ($t_c = 72$ intervals)

Key identities used in the review:

$$w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_k^{t+1} \quad (\text{FedAvg aggregation})$$

$$\hat{w}_u = w_u + b_u + \sum_{v \neq u} s_{u,v} \quad (\text{SecAgg masked update})$$

$$\sum_u \hat{w}_u = \sum_u w_u \quad (\text{pairwise masks cancel in aggregate})$$

1. Introduction

Motivation

Wind power is now central to the global energy transition: installed capacity passed 1,347 GW at the end of 2025, meeting over 11% of world electricity demand [1], and is forecast to nearly double by 2030 [2]. Operating that fleet is expensive. Operations and maintenance account for roughly 20–25% of the levelised cost of wind energy [3] and broad adoption of predictive maintenance could cut Operational and Management costs by up to 25% [4]. Better fault-detection models are therefore a direct lever on the cost of clean electricity.

The obstacle is data. Wind energy operators continuously collect rich supervisory data from their wind farms, yet this data is almost never shared across organisational boundaries, for the sake of competitive advantage.

Commercial confidentiality, competitive sensitivity, and regulatory instruments such as the General Data Protection Regulation (GDPR) [5] create hard barriers to centralising data across sites or operators. The result is a fragmented space of isolated data stores, each too small to train reliable predictive models, but collectively sufficient. It therefore becomes of great importance to be able to come up with ways which help us in training a model while preserving competitive secrets, adhering to Protection requirements and general commercial confidentiality.

Promise of Federated Learning

Federated Learning (FL) [6] was conceived precisely for this setting: a shared model is trained across distributed clients without raw data ever leaving each site.

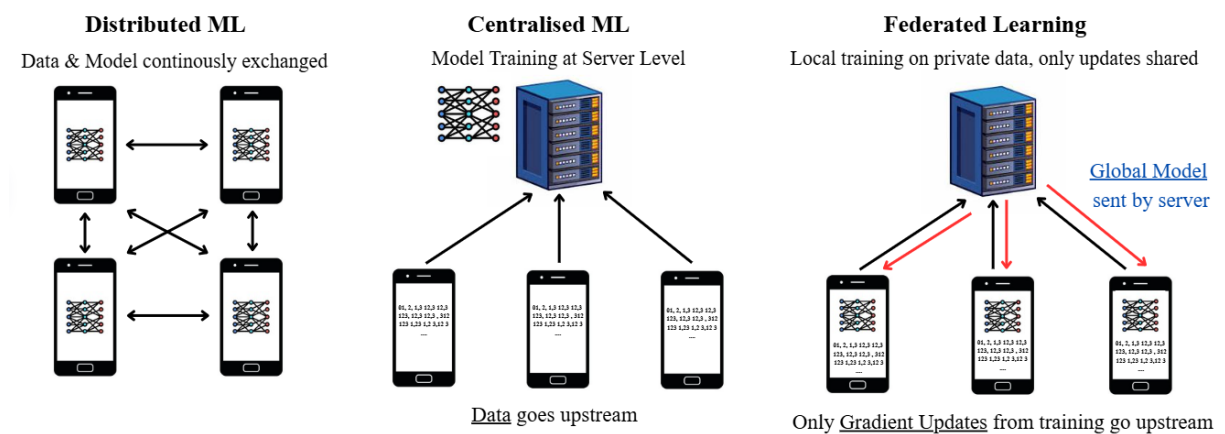


Figure 1: Contrasting Federated Learning with typical training Architectures

The protocol works by separating what each client *learns* from the data it holds. Rather than sending the data itself, each client trains the shared model briefly on its own records and transmits only the resulting adjustments to the model’s parameters: a **local gradient update** [6] that summarises what its data suggests the model should change, without the records ever leaving the site. A **central aggregator** [6] combines these updates from all

participating clients and returns a refined global model. The approach has been validated at production scale: Bonawitz et al. [7] report a deployed system handling approximately 10 million daily active devices, with up to 10,000 devices participating simultaneously in each FL round.

However, FL alone does not fully close the privacy problem. Gradient updates transmitted to the server carry enough information to permit partial reconstruction of the underlying training data. Geiping et al. [8] demonstrated that high-resolution images can be recovered from a single gradient vector using a cosine-similarity objective, without any capability beyond that of a standard **honest-but-curious** [9] aggregator (one that follows the protocol faithfully but inspects every message it receives). Protecting individual gradient updates therefore requires cryptographic guarantees that go beyond what the basic FL protocol provides.

The Remedy and the Practical Gap

The privacy vulnerabilities described above are well recognised in the literature, and several approaches have been proposed in response. McMahan et al. [6] themselves acknowledge in the original **FedAvg** paper that stronger guarantees via differential privacy, secure multi-party computation, or their combination are important directions for future work. **Differential privacy** adds calibrated noise to gradient updates to limit information leakage, but does not prevent the server from inspecting per-client transmissions, and carries an unavoidable **utility cost** (the added noise systematically reduces the accuracy of the trained model) [10] [11]. Homomorphic encryption and general secure multiparty computation offer stronger guarantees but introduce prohibitive computational and communication overhead at practical model sizes [12]. **Secure Aggregation** (SecAgg) [13] navigates these trade-offs by applying cryptographic masking selectively to the aggregation step, ensuring the server recovers only the sum of all client updates with no individual contribution ever accessible, at a communication cost linear in the number of clients. Its deployment at scale has been demonstrated [7], yet, to the author’s knowledge, the application of SecAgg to federated learning on industrial time-series regression tasks and to wind turbine **SCADA** (Supervisory Control and Data Acquisition) data in particular appears to have received limited attention in the published literature, and the privacy–utility characteristics of such a system on real-world sensor data are not yet well understood.

Review Structure

The remainder of this review is organised as follows. Section 2 establishes the formal preliminaries of federated learning and secure aggregation. Section 3 surveys the privacy threat landscape and the family of defences proposed in the literature. Section 4 examines the application of federated learning to industrial time-series settings and the structural challenges of the cross-silo regime (a small number of organisational clients holding large datasets, defined formally in Section 2.1). Section 5 synthesises the gaps identified throughout the review and formally states the research questions this thesis addresses. Section 6 draws conclusions and identifies directions for future work.

2. Preliminaries

2.1 Federated Learning

Federated Learning is a distributed training paradigm in which K clients jointly learn a shared model without exchanging raw data [6].

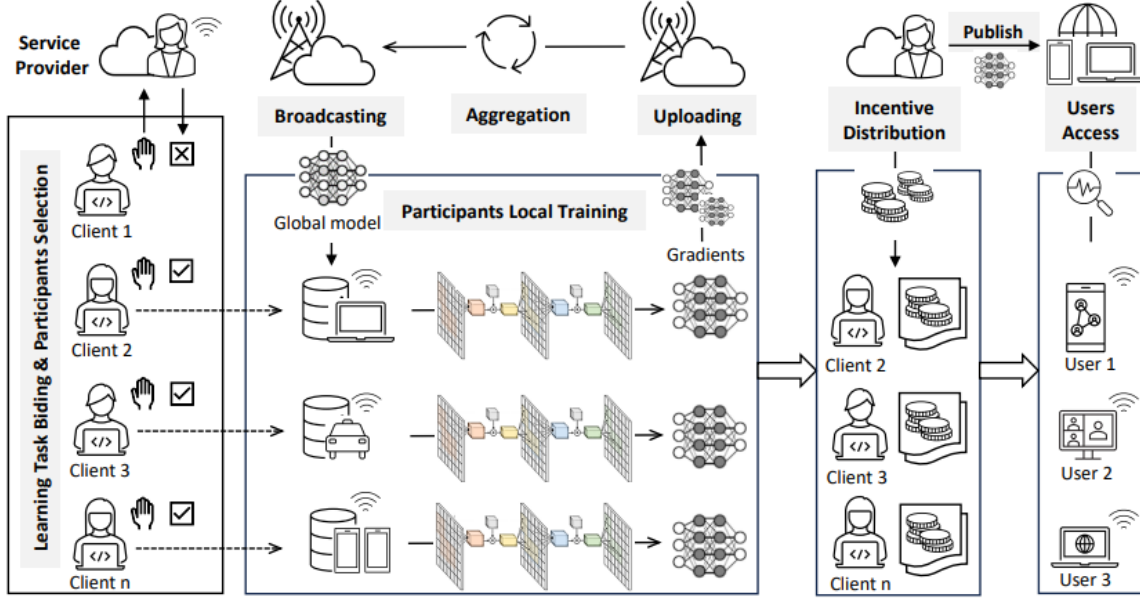


Figure 2: Federated Learning Life Cycle [14]

Mathematical Model of Federated Learning

We now formalise the per-round mechanics of the FL protocol introduced above. The model under construction is a single global predictor whose parameters live on the server; training is the repeated process of pushing these parameters out to clients, letting each client refine them on its own data, and pulling the refinements back to update the global copy. The notation below captures one such round.

Each client k holds a local dataset $\mathcal{D}_k$ of size n_k , with $n = \sum_k n_k$. In each communication round t , the server selects a subset of available clients and broadcasts the current global model w_t . Each selected client runs E epochs of local stochastic gradient descent (SGD) on $\mathcal{D}_k$ to produce a local model w_k^{t+1} , which the server then combines via the weighted average

$$w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_k^{t+1}, \quad (1)$$

known as **Federated Averaging** (FedAvg) [6]. FedAvg was shown to reduce the number of communication rounds required for convergence by 10–100 $\times$ compared to **naive federated SGD** [6], the baseline approach in which each client sends a single gradient step back to the server every round, rather than running E local epochs before synchronising (depending on the model architecture and data distribution).

Its convergence is well characterised under Independent and Identically distributed (IID) data distributions, but deteriorates under non-IID conditions: that is, when the local datasets $\mathcal{D}_k$ are drawn from systematically different distributions, a property commonly referred to as **statistical heterogeneity** [9,15]. Karimireddy et al. [16] provides a formal characterisation of this: heterogeneity in local data causes *client drift*, where local updates diverge from the global objective, leading to unstable and slow convergence.

The way this drift manifests depends on the prediction task. For **classification** problems such as wind turbine fault detection, the dominant form of heterogeneity is *label-distribution skew*: some sites log more fault events than others, and the bias accumulates principally in the final classifier layer, where each client’s local optimum reflects its own class proportions [17,18]. For **regression** problems such as continuous power-output prediction, the dominant form is *feature-distribution skew* (covariate shift): different sites see systematically different input regimes (wind speed, turbulence, air density, turbine age), so local models converge to site-specific optima that generalise poorly when averaged. Studies of FL applied to wind-power forecasting confirm this pattern empirically; large inter-farm distribution variations destabilise training and depress the accuracy of the global model, prompting clustered and personalised FL variants as mitigations [19,20]. This is directly relevant to the present setting: each wind farm’s data reflects its own physical environment, terrain, and equipment, and the regression target (power output) is the variable most sensitive to those local conditions.

Principal families of Federated Learning Approaches

To further enhance FedAvg or address the limitations of divergent training, various techniques exist. The four most prominent families are summarised below.

1. **FedProx** [15] adds a *proximal regularisation term* to each client’s local objective:

$$\frac{\mu}{2} \|w - w_t\|^2, \quad (2)$$

where w is the client’s local model during training, w_t is the global model received from the server at the start of the round, and $\mu \geq 0$ controls the strength of the penalty. The term discourages the local model w from drifting too far from the global anchor w_t , stabilising convergence under both statistical heterogeneity (non-IID data) and systems heterogeneity (clients with different compute budgets). FedProx also allows each client to perform a variable amount of local work, which is useful when clients have unequal computational resources. On the FedProx authors’ own non-IID benchmarks (MNIST, FEMNIST, Shakespeare, Sent140), FedProx improves absolute test accuracy by 22% on average over FedAvg in highly heterogeneous settings [15].

2. **SCAFFOLD** [16] takes a different approach, introducing control variates to correct for client drift directly in the gradient updates. SCAFFOLD is provably unaffected by data heterogeneity and client sampling, and requires significantly fewer communication rounds than FedAvg to reach the same accuracy.

3. **FedMA** [21] (Federated Matched Averaging) replaces FedAvg’s coordinate-wise weight averaging with a layer-wise matching procedure: hidden units across clients (channels in CNNs, hidden states in LSTMs, neurons in fully-connected layers) are first permuted to align units with similar feature-extraction signatures, then averaged. This addresses the permutation invariance of neural network parameters, which can cause naive averaging to destroy learned structure, and is particularly effective for deeper CNN and LSTM architectures.
4. **Personalised FL** approaches, such as FedPer [22], depart from the goal of a single shared global model. Instead, the network is split into a *base* portion (the lower layers, typically the feature extractor) that is shared and aggregated across clients in the usual federated manner, and a *personalisation* portion (the upper layers, typically the task-specific head) that each client trains and keeps locally. This lets every client benefit from collaborative representation learning while retaining a model tuned to its own data distribution, which is helpful when client distributions differ enough that no single global model fits all of them well. Knowledge-distillation variants extend the same idea by sharing prediction outputs rather than parameters, accommodating architecturally heterogeneous clients [12].

The appropriate choice among these families is governed primarily by the degree of data heterogeneity present and the computational constraints of the deployment.

Deployment Methods for Federated Learning

Two deployment regimes are distinguished in the literature [9, 12].

1. **Cross-device** FL involves large numbers of clients, typically 10^3 to 10^7 , such as mobile phones or IoT sensors that participate intermittently, hold small local datasets, and are effectively anonymous.
2. **Cross-silo** FL involves a small number of identifiable organisational clients, typically 2 to 100 entities such as hospitals or industrial operators, that are reliably available and each hold substantial datasets.

Wind farm operators correspond to the *cross-silo regime*: a small number of competing companies, each with a large fleet of instrumented turbines.

This distinction carries significant implications for protocol design. Most published results, including the SecAgg protocol and its production deployment, were developed and evaluated in the cross-device regime, where K is large enough that statistical properties of the protocol behave predictably. Liu et al. [23] note explicitly that prior work on privacy-preserving FL has focused on cross-device settings whose dominant challenges (efficiency at large K , resilience to frequent client dropout) are **not** the dominant concerns of the cross-silo regime, where participant count is small but stringent confidentiality requirements apply. At the small K values characteristic of the cross-silo wind energy setting, the behaviour of SecAgg and related protocols is therefore comparatively less characterised,

a point revisited in Section 5.

Key Challenges with Federated Learning

1. **Costly communication.** Sharing large model parameter vectors across many rounds is expensive, particularly when clients have limited upload bandwidth. As noted earlier in this section, FedAvg itself was introduced partly to address this, reducing the required number of communication rounds by 10–100× over naive federated SGD by running E local epochs per round before synchronising [6]. Complementary strategies include gradient quantisation and selective parameter transmission.
2. **System heterogeneity.** Variation in clients’ hardware, network conditions, and computational resources causes stragglers and dropout during training rounds. As discussed under *Principal families of Federated Learning Approaches* earlier in this subsection, FedProx [15] partially addresses this by tolerating variable amounts of local work per client.
3. **Statistical heterogeneity (non-IID data).** Local datasets are rarely identically distributed across clients. This is the most consequential challenge for model quality, and the one most directly relevant to this thesis, where site-specific wind conditions create systematic distributional shift across turbine operators.
4. **Privacy concerns.** Even though raw data never leaves the client, gradient updates carry information about the underlying data, exposing clients to inference attacks from an honest-but-curious server. Addressing this challenge is the primary motivation for Secure Aggregation, introduced in Section 2.2.

2.2 Secure Multiparty Computation and Secure Aggregation

The aggregation step in FedAvg is widely identified as the principal privacy vulnerability in standard FL [13, 14]: the server receives each client’s gradient update individually and can inspect it before aggregating. Cryptographic techniques from Secure Multiparty Computation (SMC) address this directly: they allow multiple parties to jointly compute a function over their private inputs without any party revealing those inputs to the others.

The theoretical foundations rest on two classical results. Shamir [24] introduced *secret sharing*: a value s is encoded into K shares such that any t shares suffice to reconstruct s , while any $(t - 1)$ shares reveal nothing. Ben-Or et al. [25] established the general feasibility result that any polynomial-time function can be computed securely under this model by K parties, provided fewer than $K/3$ are actively malicious. Together, these results guarantee that the FL aggregation sum $\sum_k w_k$ can, in principle, be computed without the server ever observing any individual w_k .

Overview of the Secure Aggregation Setup

Bonawitz et al. [13] translated these foundations into a practical Secure Aggregation (SecAgg) protocol designed specifically for the FL setting. The protocol proceeds in four communication rounds, each with $O(K)$ communication cost per client, summarised in Table 1. The central mechanism is additive masking: each client u masks its update with a sum of pairwise pseudorandom values $s_{u,v}$, agreed with each other client v via Diffie–Hellman (DH) key exchange, together with a private self-mask b_u . Each client therefore submits

$$\hat{w}_u = w_u + b_u + \sum_{v \neq u} s_{u,v}. \quad (3)$$

The pairwise masks are constructed to cancel in the aggregate, so that

$$\sum_u \hat{w}_u = \sum_u w_u, \quad (4)$$

while no individual w_u is ever accessible to the server. Client dropout is tolerated through secret sharing of the mask values: if a client drops out after committing its masked update, the server can recover the corresponding masks from the shares held by other surviving clients. If too many clients drop out before committing, the round is aborted entirely rather than revealing individual updates.

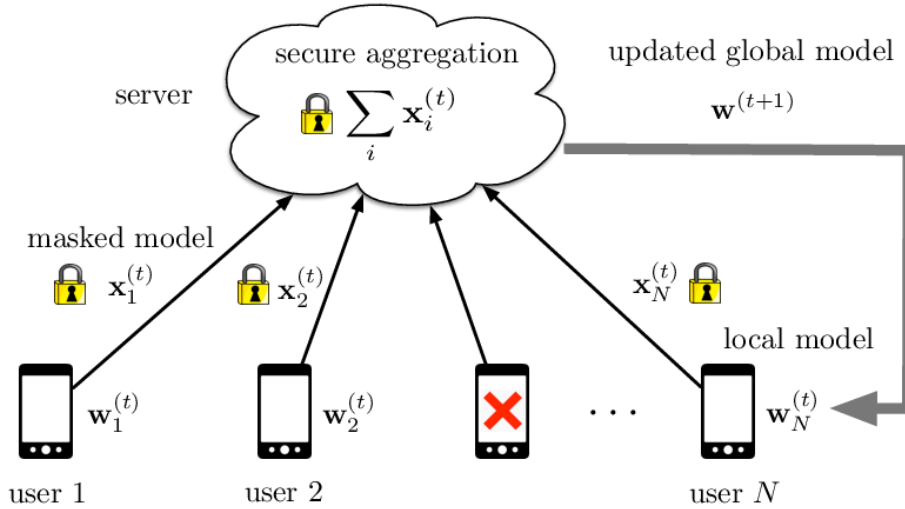


Figure 3: Conceptual overview of Secure Aggregation: each client transmits a masked update to the server, which recovers only the sum of all updates. Adapted from Bonawitz et al. [13].

Figure 4 visualises the same protocol as a message-flow diagram, showing the order in which keys, shares, and masked updates are exchanged between a representative client u , the other participating clients, and the server.

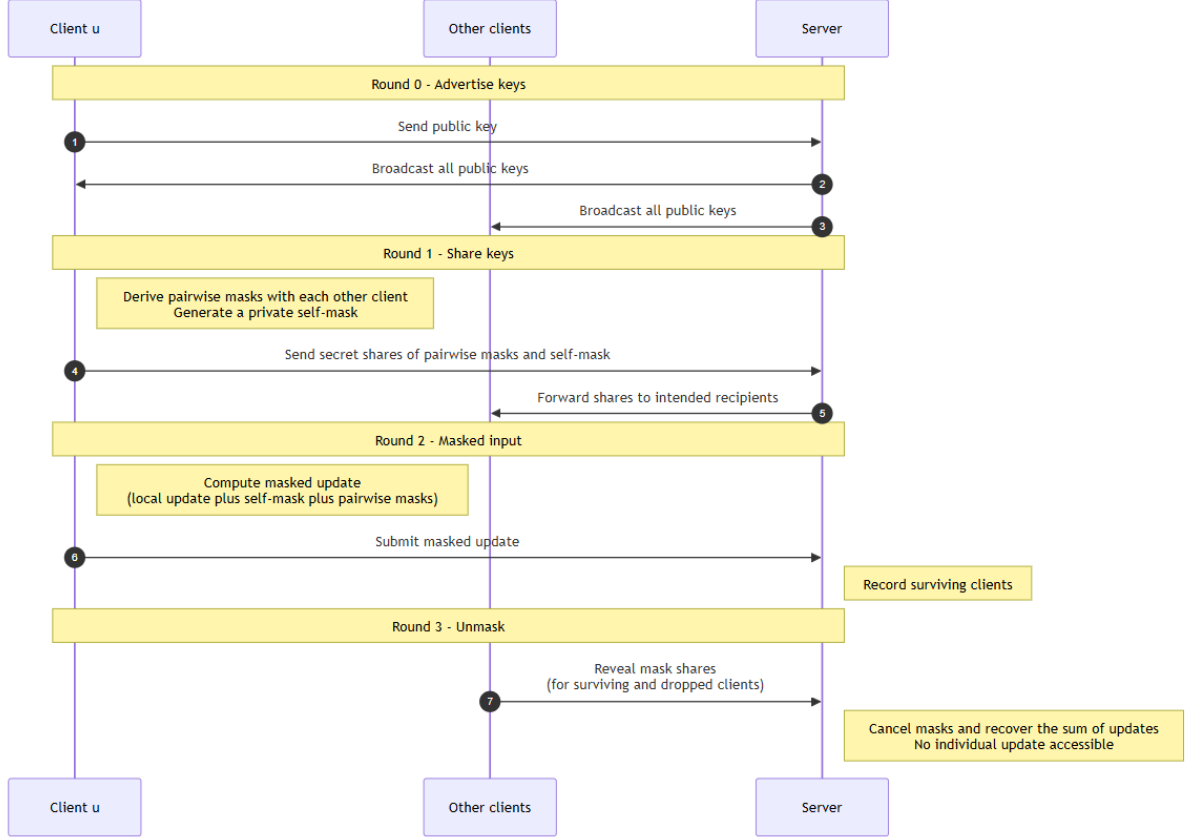


Figure 4: Message flow of the SecAgg protocol across its four communication rounds. Each client u first establishes pairwise masks $s_{u,v}$ with every other client v via Diffie–Hellman key exchange (Rounds 0–1), then submits a masked update $\hat{w}_u = w_u + b_u + \sum_{v \neq u} s_{u,v}$ to the server (Round 2). The pairwise masks cancel in aggregate, allowing the server to recover $\sum_u w_u$ without observing any individual w_u (Round 3). Dropout-handling logic, in which the server reconstructs the masks of departed clients from secret shares held by surviving clients, is omitted for clarity; the full specification is given in Bonawitz et al. [13].

Table 1: SecAgg protocol phases, client and server actions, and per-round communication cost (K = number of clients). Adapted from Bonawitz et al. [13].

Round	Phase	Client action	Server action
0	Advertise keys	Generate DH key pair and transmit public key	Collect and broadcast all public keys
1	Share keys	Derive pairwise masks and self-mask, distribute secret shares of both	Forward shares to intended recipients
2	Masked input	Submit masked update $\hat{w}_u = w_u + b_u + \sum_{v \neq u} s_{u,v}$	Accumulate masked inputs and record surviving clients
3	Unmask	Reveal mask shares for surviving and dropped clients	Cancel masks and recover $\sum_u w_u$

All four rounds carry $O(K)$ communication cost per client. If insufficient clients survive Round 2, the protocol aborts and no individual update is revealed.

Scaling Considerations

Vanilla SecAgg does not scale well. Because every client must establish pairwise masks with every other client, the server-side computation and the per-client key-sharing cost both grow as $O(K^2)$ in the number of participants. Two practical mitigations have been proposed.

The first, used in the production deployment of Bonawitz et al. [7], partitions clients into smaller subgroups and runs SecAgg independently within each subgroup. A master aggregator then sums the resulting partial sums without further protection, since each partial sum is already a masked aggregate from which no individual update can be recovered.

The second, introduced by Bell et al. [26] and known as SecAgg+, replaces the all-pairs key-exchange topology with a sparse random graph: each client only shares secrets with a logarithmic-sized neighbourhood rather than the full client set. This reduces the key-sharing cost from $O(K^2)$ to $O(K \log K)$ while preserving the same privacy guarantee under the same threat model. The Flower framework [27] provides an open-source implementation of SecAgg+ [28].

3. Privacy in Federated Learning: Threats and Defences

3.1 Threat Landscape

Overview

The privacy problem in federated learning is not resolved by keeping raw data local. Gradient updates transmitted to the aggregator carry statistical information about the data from which they were computed, and several classes of attack exploit this residual signal. The threats against the FL life cycle can be organised into two broad families: *utility-harmed* attacks, which degrade model accuracy or robustness, and *privacy-harmed* attacks, which compromise the confidentiality of participants' data [14]. Li et al. [14], in a comprehensive 2024 survey, demonstrate that such threats are not confined to the local training phase but can emerge at any stage of FL execution, from participant selection through to model publication and user access. Guendouzi et al. [12] similarly identify four categories most pertinent in FL deployments: data leakage via gradient inversion, membership inference, model poisoning, and the structurally fundamental honest-but-curious server. All four are relevant to the cross-silo industrial setting, though their relative severity differs from the cross-device case.

FL Life Cycle		Client Selection	Model Broadcast	Local Training	Aggregation	Incentive	Users Access
Actor Interacted Surface		Server, Potential clients Cloud, Ground	Server Cloud, Network	Clients Ground, Network	Sever Cloud	Server, Clients Cloud, Ground	Users Ground
Utility	Poisoning	●	●	○	●	●	●
	Promise Breaking	○ ●	●	●	●	●	●
	Sybil	○	●	○	●	●	●
	Communication	●	○ ●	●	●	●	●
	Evasion	●	●	●	●	●	○
Privacy	Inference	■	■	□	□	■	□
	Inversion	■	■	■	□ ■	■	■
	Eavesdropping	■	□	□	■	■	■

● ■ Non-exist, ○ ■ Exist and Potential impact, ○ □ Exist and Direct impact

Figure 5: Vulnerabilities throughout the FL life cycle and their impacts. Each cell indicates whether a given threat is non-existent, has potential impact, or has direct impact at the corresponding pipeline stage. Adapted from Li et al. [14].

1. Honest-but-curious server

The most structurally fundamental threat is an aggregator that follows the protocol faithfully but records and inspects all received gradient vectors Δw_k . Because each update is computed from a single client's private dataset, its inspection gives the server direct access to per-client information. This threat is inherent to the standard FL protocol and is not mitigated by local data retention alone. Bonawitz et al. [7] explicitly identify this as one of the threats SecAgg is designed to counter, ensuring that individual devices' updates remain encrypted even in-memory within the aggregator infrastructure.

2. Gradient inversion

The severity of gradient exposure was placed on a quantitative footing by Geiping et al. [8], who showed that training images can be reconstructed from a single gradient vector with high fidelity. The attack frames reconstruction as an optimisation problem: given the observed gradient update Δw_k , the attacker searches over candidate inputs x' for the one whose gradient most closely matches Δw_k in direction, measured via cosine similarity. Writing the per-sample loss as $\mathcal{L}$ and the recovered estimate of the original input as $\hat{x}$, this is

$$\hat{x} = \arg \min_{x'} \left(1 - \frac{\nabla \mathcal{L}(x') \cdot \Delta w_k}{\|\nabla \mathcal{L}(x')\| \|\Delta w_k\|} \right), \quad (5)$$

which requires no capability beyond observing a standard FedAvg gradient update. Li et al. [14] classify inversion attacks as *privacy-harmed* threats with both potential and direct impact during the local training and aggregation phases, confirming that they represent a persistent vulnerability throughout the FL life cycle. Averaging gradients over multiple iterations or images does not provide meaningful protection [8]. This establishes gradient inversion as a realistic, not merely theoretical, threat.

3. Membership inference

Membership inference attacks exploit the tendency of models to behave differently on training versus non-training data, allowing an adversary to determine whether a specific record belonged to a particular client’s dataset [29]. Unlike gradient inversion, this attack targets the trained model rather than the raw gradient updates, and persists even when SecAgg is applied. Li et al. [14] note that TEEs and decentralised learning schemes represent emerging directions for privacy-harmed attack defence, though neither has achieved the deployment maturity of SecAgg. A related strand of recent work explores blockchain-based FL, in which a distributed ledger replaces the central aggregator and audit trails of model updates are made tamper-evident; this addresses trust-establishment between competing organisational clients but introduces its own scalability and energy-cost trade-offs [30]. All three directions are noted here for completeness.

4. Byzantine and poisoning attacks

Malicious clients may submit deliberately crafted updates to steer the global model toward a targeted behaviour (model poisoning) or degrade overall performance (Byzantine attack) [9]. These are active rather than passive threats and require separate mitigations: Byzantine-robust aggregation rules such as coordinate-wise median or Krum [9]. Li et al. [14] further identify promise breaking and Sybil attacks as utility-harmed threats at the participant selection phase, where dishonest clients misrepresent their capabilities to gain selection. Active threat classes are noted here for completeness; defending against them is an orthogonal problem to aggregation privacy and is generally treated as a separate line of work in the FL literature [12].

Table 2 summarises these threat classes and their primary mitigations.

Table 2: Threat taxonomy for federated learning systems. All threats below except model poisoning assume a *semi-honest* adversary that follows the protocol faithfully but inspects every message it receives; model poisoning assumes an *active* adversary that may deviate from the protocol. Severity is rated relative to a cross-silo regression deployment with an honest-but-curious server. Life-cycle phase classification adapted from Li et al. [14].

Threat	Mechanism	Severity	Primary mitigation
Honest-but-curious server	Inspects per-client gradient updates Δw_k directly	High	Secure Aggregation [13]
Gradient inversion [8]	Reconstructs inputs by matching candidate gradients to the observed update via cosine similarity (Eq. 5)	High	Secure Aggregation [13] or Differential Privacy [31]
Membership inference [29]	Infers record membership from differences in model behaviour on training versus non-training data	Medium	Differential Privacy [10]
Model poisoning [9]	Crafted updates steer or degrade the global model	Medium	Byzantine-robust aggregation [9]

Foundations

Differential Privacy (DP), introduced by Dwork and Roth [10], is the dominant statistical framework for formal privacy guarantees in machine learning. Informally, a mechanism is differentially private if its output distribution is nearly indistinguishable whether or not any single record is included in the input: an observer looking at the result learns essentially nothing new about that individual record. Formally, a randomised mechanism $\mathcal{M}$ satisfies (ϵ, δ) -DP if for any two adjacent datasets D and D' differing in one record, and for any output set S ,

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \Pr[\mathcal{M}(D') \in S] + \delta. \quad (6)$$

Intuitively, the exponential factor e^ϵ caps how much more likely any particular output becomes when an individual record is added or removed; the additive δ allows a small probability of failure for the guarantee. The parameter $\epsilon \geq 0$ is the *privacy budget*: smaller values impose stronger privacy at a greater cost to model utility, and pure ϵ -DP corresponds to $\delta = 0$.

Noise mechanisms

Two mechanisms are canonical. The *Laplace mechanism* adds Laplace-distributed noise calibrated to the L_1 sensitivity of the query (the maximum change in the query output, measured as the sum of absolute differences, when one record is added or removed) [10]. It is best suited to scalar outputs. The *Gaussian mechanism* adds Gaussian noise calibrated to the L_2 sensitivity (the same notion but measured as the Euclidean norm of the change), and is more amenable to composition over many queries [10]. Abadi et al. [31] applied the Gaussian mechanism to deep learning via the DP-SGD algorithm: gradients are clipped

to a bounded L_2 norm C and Gaussian noise drawn from

$$\mathcal{N}(0, \sigma^2 C^2) \quad (7)$$

is added before aggregation, where σ is the noise multiplier and the product σC controls the standard deviation of the added noise. Clipping bounds the influence of any single sample, and the calibrated noise then masks that influence in the aggregate. Abadi et al. also introduced the *moments accountant*, a tight composition method for tracking the accumulated privacy cost ε over many training steps.

Local, central, and shuffled Differential Privacy

In *local* DP, each client adds noise before transmitting its update, protecting against a fully untrusted server at the cost of higher noise for the same privacy budget. In *central* DP, the aggregator adds noise to the aggregate after collection, permitting lower noise because the sensitivity of the sum is lower than the sensitivity of individual updates [9, 10]. A third variant, *shuffled* DP, interposes an anonymous shuffler between clients and server to amplify the privacy guarantee of local DP without requiring a fully trusted aggregator [9].

Rényi Differential Privacy and privacy accounting

A more general accounting framework, introduced by Mironov [32], uses Rényi divergence rather than the max-divergence implicit in the standard (ε, δ) -DP definition. The *max-divergence* measures the worst-case ratio of output probabilities under adjacent datasets and corresponds to the strict multiplicative bound in Eq. 6; *Rényi divergence* generalises this to a family of moment-based bounds parameterised by an order $\alpha > 1$, recovering max-divergence as a limiting case. Rényi DP subsumes (ε, δ) -DP and provides cleaner composition rules [9, 32], enabling tighter privacy budget tracking over long training runs. In practice, libraries such as TensorFlow Privacy implement Rényi-based accounting to compute the exact (ε, δ) guarantee at any training step.

Limitations of Differential Privacy alone

Critically, DP does not prevent the server from receiving and inspecting per-client gradient updates. Local DP perturbs what the server sees but does not prevent access; central DP adds noise only to the aggregate, which the server has already seen in full. Liu et al. [11] confirm across 147 FL papers that stronger privacy always causes greater information loss, a trade-off that is unavoidable. The shape of this trade-off is illustrated empirically in Figure 6, reproduced from Abadi et al. [31]: on MNIST under DP-SGD, weakening the privacy budget from $\varepsilon = 0.5$ through $\varepsilon = 2$ to $\varepsilon = 8$ recovers test accuracy from $\sim 90\%$ to $\sim 95\%$ to $\sim 97\%$, against an unprotected baseline of $\sim 99\%$.

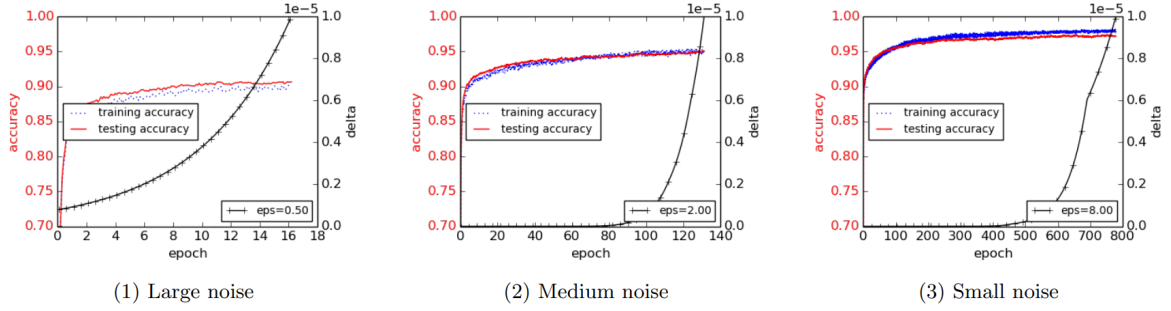


Figure 6: The privacy–utility trade-off in DP-SGD, illustrated on MNIST. Training and test accuracy are plotted over training epochs under three noise levels, yielding final (ϵ, δ) guarantees of $(0.5, 10^{-5})$, $(2, 10^{-5})$, and $(8, 10^{-5})$ at test accuracies of approximately 90%, 95%, and 97% respectively (non-private MNIST achieves $\sim 99\%$). Stronger privacy (smaller ϵ) imposes a monotonically larger utility cost. Reproduced from Abadi et al., *Deep Learning with Differential Privacy*, CCS 2016 [31], Figure 3.

For regression tasks such as SCADA power output prediction, where prediction error has direct operational consequences, this trade-off requires careful calibration. SecAgg resolves the access problem entirely by preventing individual gradient inspection, making it complementary to DP rather than interchangeable.

3.2 Cryptographic Defences

Overview

Three families of cryptographic defence have been applied to FL: Secure Aggregation, Homomorphic Encryption (HE), and general Secure Multiparty Computation (SMC) [12, 14]. They differ substantially in privacy guarantees, computational costs, and applicability to regression tasks. Li et al. [14] identify all three, alongside Trusted Execution Environments and decentralised learning schemes, as the principal defence categories against privacy-harmed attacks in FL systems.

Secure Aggregation

SecAgg provides *aggregation privacy*: as introduced in Section 2.2, the server recovers only the sum

$$\hat{w} = \sum_{k=1}^K w_k \quad (8)$$

and cannot access any individual w_k . This directly addresses both the honest-but-curious server threat and gradient inversion. The $O(K)$ per-client communication overhead is modest and the protocol’s native dropout tolerance makes it well suited to settings where client availability is not guaranteed. In practice, the quadratic server-side cost (the $O(K^2)$ cost of pairwise key agreement and mask reconstruction) is the dominant scalability bottleneck, and is addressed in the production deployment of Bonawitz et al. [7] by running SecAgg within individual aggregator instances over subgroups of clients, with a master aggregator combining partial sums without SecAgg. The SecAgg+ variant of Bell et

al. [26] reduces this to $O(K \log K)$ via a sparse random graph structure, as discussed in Section 2.2.

Homomorphic Encryption

Homomorphic Encryption (HE) supports arithmetic operations directly on ciphertext, allowing the server to aggregate encrypted updates and return an encrypted global model. Fully Homomorphic Encryption supports arbitrary computation but carries extreme computational overhead. Partial and levelled HE schemes support limited operations at lower cost, but cannot straightforwardly handle nonlinear activation functions without approximation error [11, 12]. Even with the most efficient practical scheme (CKKS), HE-based FL has been measured to inflate per-round communication by roughly $17\times$ compared to unprotected FedAvg [33]. This overhead has direct operational consequences in industrial settings: predictive-maintenance models on wind turbine SCADA must respond within minutes of a fault onset to be useful, and the cumulative latency of fully homomorphic computation over deep models is often incompatible with such real-time constraints. The multi-round structure of SecAgg also adds latency, but in the cross-silo regime with few clients the round-trip cost is bounded and predictable rather than scaling with K . Liu et al. [23] propose a doubly homomorphic secure aggregation scheme for cross-silo FL using multi-key HE, representing a maturing but not yet production-ready alternative to SecAgg.

General Secure Multiparty-Computation and Trusted Execution Environments

General SMC protocols guarantee that any polynomial-time function can be computed securely [25]. Practical frameworks include SecureML [34] for two-party privacy-preserving regression, but communication complexity grows as $O(K^2)$ in the number of clients, making general SMC impractical at even moderate scale [11, 12]. Trusted Execution Environments (TEEs) such as Intel SGX offer a hardware-enforced alternative with low computational overhead, but require specific infrastructure and are vulnerable to side-channel attacks. Li et al. [14] note that TEEs and decentralised learning schemes represent emerging directions for privacy-harmed attack defence, though neither has achieved the deployment maturity of SecAgg. They are noted here for completeness. Table 3 consolidates the comparison across all mechanisms, ordered by approximate per-client communication overhead.

Table 3: Privacy-preserving FL mechanisms, ordered by per-client communication overhead. Defence categorisation informed by Li et al. [14].

Mechanism	Guarantee	Comm.	Comp.	Dropout	Regr.	Example
DP local [10]	(ϵ, δ) -DP per client	$O(1)$	$O(d)$	Yes	Yes	—
DP central [31]	(ϵ, δ) -DP on aggregate	$O(1)$	$O(d)$	Yes	Yes	DP-SGD [31]
TEE	Hardware isolation	$O(1)$	Low	Yes	Yes	Intel SGX
SecAgg+ [26]	Aggregation privacy	$O(\log K)$	$O(\log^2 K)$	Yes	Yes	Flower [28]
SecAgg [13]	Aggregation privacy	$O(K)$	$O(K^2)$	Yes	Yes	Bonawitz et al. [7]
SecAgg + DP	Aggregation privacy + (ϵ, δ) -DP	$O(K)$	$O(K^2)$	Yes	Yes	Bonawitz et al. [7]
HE (partial / CKKS)	Ciphertext computation	$\sim 17\times$	High	No	Limited	Liu et al. [23]
General SMC [25]	Arbitrary function privacy	$O(K^2)$	High	No	Yes	SecureML [34]

Comm./Comp. report per-client communication and server-side computation cost in the number of clients K (where d is the model dimension). Dropout indicates native support for client dropout. Regr. indicates applicability to continuous prediction without fundamental obstacles. HE entries report measured overhead as a multiple of unprotected FedAvg communication, since asymptotic per-client cost varies by scheme.

3.3 Practical Deployments and Hybrid Approaches

Production deployment

The at-scale deployment documented by Bonawitz et al. [7] remains the most comprehensive engineering reference for FL with SecAgg in production. The system is built around an actor model: **Coordinators** manage round progression; **Selectors** accept and forward device connections; and **Master Aggregators** manage round execution and spawn child aggregators as needed. All actor state is kept in-memory and ephemeral, with no per-device updates written to persistent storage, which itself removes a class of attacks targeting persistent logs. Client dropout rates between 6% and 10% are observed consistently, with the server selecting 130% of the target device count to compensate.

The interaction between these actor types within a single training round is shown in Figure 7.

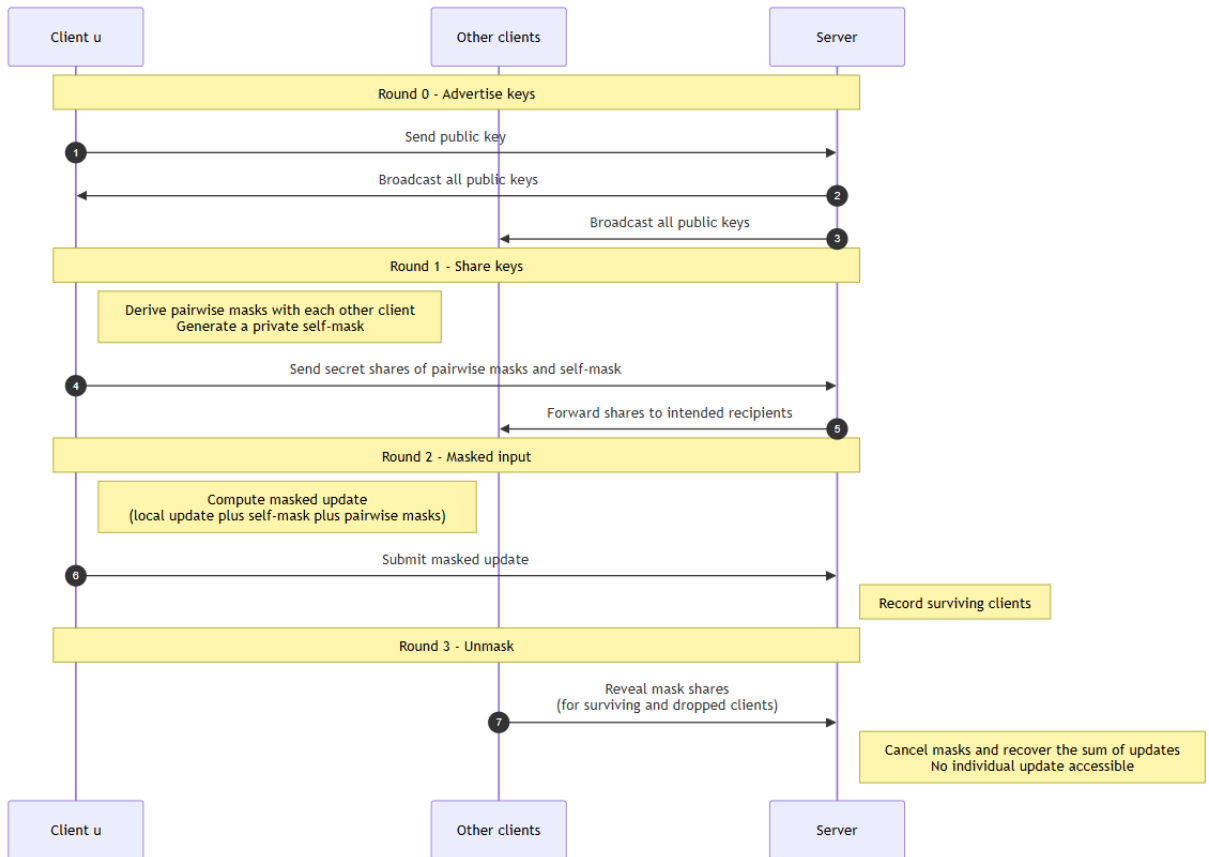


Figure 7: Message flow within a single training round of the production FL system of Bonawitz et al. [7]. The *Coordinator* drives round progression; *Selectors* admit and forward incoming device connections; and the *Master Aggregator* spawns child aggregators that run SecAgg over subgroups of devices and combines the resulting masked partial sums. All state is kept in-memory and ephemeral. Selection logic targeting 130% of the desired participant count (to absorb the 6–10% client dropout rate observed in production) and dropout-handling within child aggregators are omitted for clarity.

Hybrid Differential Privacy and Secure Aggregation

The combination of SecAgg with DP provides complementary guarantees: SecAgg prevents individual gradient access while DP bounds the information in the aggregate. In this hybrid, each client applies DP noise to its gradient before the SecAgg masking step, so the server receives only a masked sum of already-noisy gradients. McMahan et al. [35] demonstrate this combination for differentially private recurrent language models, showing that useful models can be trained under meaningful privacy budgets. Bonawitz et al. [7] confirm it is now the state-of-the-practice in production FL systems.

Framework support

Guendouzi et al. [12] compare 11 FL frameworks and confirm that, to the author’s knowledge, only Flower [27] natively integrates SecAgg+ as a configurable component. The other major frameworks surveyed (TensorFlow Federated, IBM FL, FedML, PySyft) rely on DP or SMC as their primary privacy mechanism without built-in SecAgg support. The interaction between SecAgg and DP in a regression setting with non-IID time-series data has not, to the author’s knowledge, been systematically studied in the published literature, and constitutes one of the open questions identified in Section 5.

4. Federated Learning for Industrial Time-Series

4.1 Characteristics of Industrial Sensor Data

Overview

Industrial sensor data presents a set of structural characteristics that distinguish it from the image, text, and tabular benchmarks on which most FL research has been evaluated. Understanding these characteristics is essential before assessing the applicability of existing FL algorithms and privacy mechanisms to the industrial setting.

Non-IID distributions across sites

Industrial sites differ in equipment model, age, terrain, operating regime, and maintenance practice, creating systematic distributional shift across federated clients. This is exactly the form of statistical heterogeneity discussed in Section 2.1 under *Principal families of Federated Learning Approaches*, where regression tasks were shown to suffer predominantly from feature-distribution skew (covariate shift) rather than the label-distribution skew typical of classification. The heterogeneity is structural rather than incidental: the same physical quantity (e.g. active power output) measured at two different wind farms will follow different distributions because the local wind resource, turbine geometry, and control firmware differ. Liu et al. [11] synthesise evidence across 147 FL papers showing that non-IID data can cause FL to fail to match the accuracy of centralised training, whereas under IID conditions the gap is typically within 3%; this IID-vs-non-IID accuracy gap is shown empirically in Figure 7.

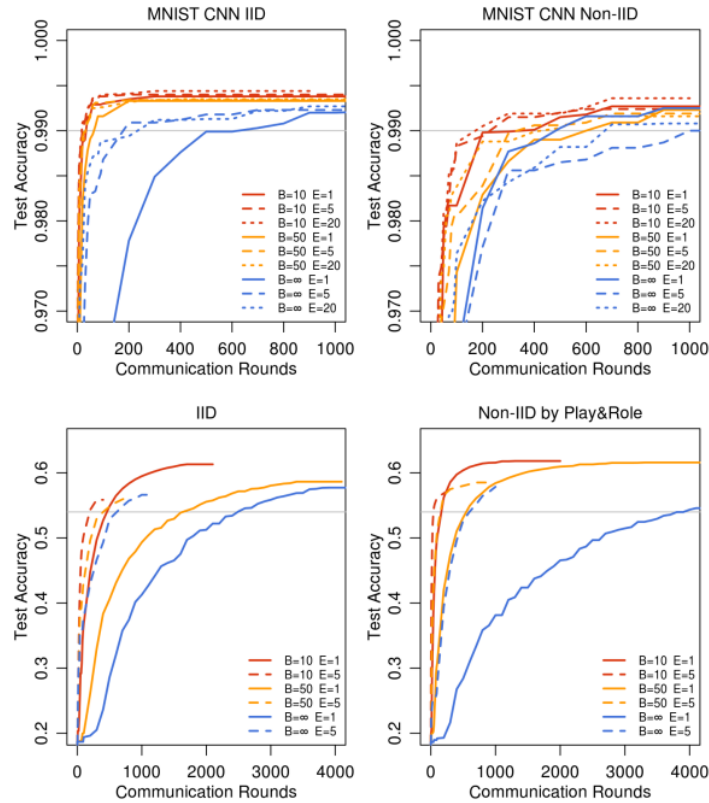


Figure 7: Test accuracy versus communication rounds for FedAvg on two canonical benchmarks: MNIST CNN (top) and Shakespeare LSTM (bottom), under IID partitioning (left) and pathological non-IID partitioning (right). The gap between client-batch configurations (B, E) narrows considerably under IID and widens under non-IID, illustrating that statistical heterogeneity is the dominant degradation factor in FL convergence. Reproduced from McMahan et al., *Communication-Efficient Learning of Deep Networks from Decentralized Data*, AISTATS 2017 [6], Figure 2.

Feature heterogeneity

Different equipment models at different sites expose different sensor channels. A feature present in one site’s schema may be absent, differently named, or differently calibrated at another. This imposes a feature-alignment requirement before horizontal FL can proceed, restricting the usable feature set to the intersection of available channels across all participating clients. To the author’s knowledge, no standard protocol exists in the FL literature for negotiating this alignment automatically across organisational boundaries, and the issue is typically handled by ad-hoc preprocessing per deployment.

Temporal autocorrelation

Industrial sensor streams are multivariate time series with strong serial dependence: measurements at time t are highly predictive of measurements at $t + 1$. This violates the IID sample assumption underlying FedAvg’s convergence analysis and the shuffled mini-batch setting for which DP-SGD was designed. The interaction between local temporal autocorrelation and global cross-site distributional heterogeneity has not, to the author’s knowledge, been characterised for regression tasks.

Irregular availability and data quality

Industrial equipment goes offline for scheduled maintenance, fault events, and curtailment, making client participation in FL rounds unreliable even in the cross-silo regime. Additionally, industrial sensor data commonly suffers from quality issues: missing values due to sensor dropout, erroneous readings from faulty sensors, and timestamp gaps. These must be addressed in preprocessing and constrain which FL communication protocols are viable.

Commercial sensitivity

Industrial sensor data reflects proprietary operating procedures, equipment health, and production performance. Operators have strong commercial incentives to withhold it from competitors, and the GDPR [5] creates legal barriers to centralisation across organisational boundaries. These factors together produce the data-silo structure that FL is designed to address.

4.2 Wind Turbine SCADA: Structure and the CARE Dataset

This subsection grounds the abstract characteristics above in the concrete data substrate used by this thesis. It first describes the structure of SCADA data as it is recorded in modern wind farms, then introduces the CARE benchmark dataset of Gück et al. [36], which provides the multi-farm SCADA corpus on which the empirical work is conducted.

SCADA structure

Wind turbine Supervisory Control and Data Acquisition (SCADA) systems log operating state at 10-minute intervals across a range of sensor channels. These commonly include rotor speed, active and reactive power output, pitch angle of each blade, nacelle orientation (yaw), generator winding temperatures, gearbox temperatures, and ambient wind speed and direction [37]. The resulting multivariate time series encode both normal operating behaviour and the gradual onset of mechanical or electrical faults, making them valuable for predictive maintenance.

The multi-operator structure of the wind energy industry maps directly onto the FL cross-silo regime introduced in Section 2.1. A wind farm operator holds SCADA data for their own fleet but has no access to data from competing operators. Collaborative model training across operators would allow all parties to benefit from a larger effective training corpus, improving fault detection accuracy and reducing unplanned downtime, without any operator exposing commercially sensitive telemetry. The GDPR [5] and analogous instruments make this framing not merely convenient but legally necessary in many jurisdictions.

The CARE dataset

The empirical work in this thesis uses the CARE (Coverage, Accuracy, Reliability and Earliness) dataset of Gück et al. [36]. CARE was constructed to benchmark fault detection algorithms along four dimensions: coverage of the turbine fleet, accuracy of fault identification, reliability of confirmed detections, and earliness of warning. These dimensions make it a more demanding benchmark than datasets evaluated solely on classification accuracy.

The dataset comprises SCADA records from 36 turbines across three wind farms, providing natural federated partitions at two granularities: farms ($K = 3$ clients) or turbines ($K \leq 36$ clients). The 10-minute resolution and 95 CSV files provide a realistically large corpus for time-series regression. A criticality threshold $t_c = 72$ (72 intervals $\times$ 10 minutes = 12 hours of continuous fault status) determines which events are classified as actionable. Documented quality issues, including NaN-valued intervals, sensor dropout events, and status-code gaps, are representative of real-world industrial data and must be addressed in preprocessing. Table 4 summarises the key properties.

Table 4: Key properties of the CARE wind turbine SCADA dataset of Gück et al. [36].

Property	Value
Wind farms	3
Turbines	36
CSV files	95
Temporal resolution	10 minutes
Criticality threshold t_c	72 intervals (12 hours)
Federated partition options	Per farm ($K = 3$) or per turbine ($K \leq 36$)
Key sensor channels	Active power, rotor speed, pitch angle, nacelle temperature, generator temperature, vibration
Known quality issues	NaN-valued intervals, sensor dropout, status-code gaps

4.3 Federated Learning Applied to Industrial and Wind Energy Domains

The industrial FL landscape

The systematic review by Liu et al. [11], covering 147 FL papers from 2016 to early 2020, found that industrial manufacturing accounted for approximately 4% of papers, by far the smallest share of any application domain. Edge computing and IoT (33%), healthcare (29%), and urban computing (17%) dominated. The industrial problems addressed were topic modelling, environment condition monitoring, product visual inspection, and sensor failure prediction. Wind energy appeared in none of them. Guendouzi et al. [12], reviewing 84 FL survey papers between 2019 and 2023, confirm that despite overall FL publication volume growing from approximately 75 papers in 2019 to over 2,700 by 2023, industrial sensor regression with cryptographic privacy remained, to the author’s knowledge, largely absent from the surveyed academic literature. Some applied work may appear in venues less systematically indexed by survey papers, notably the Applied Data Science track of KDD [38]; this track explicitly solicits deployed industrial systems, and the KDD Cup 2022 wind power forecasting challenge of Zhou et al. [39] attracted over 2,400 teams working on SCADA-based wind power forecasting. None of the top-placed solutions were federated, however, indicating that even within this applied community the FL-with-cryptographic-privacy combination remains under-explored.

Post-2020 wind energy FL work

Notwithstanding the absence noted above in cross-cutting surveys, the period since 2020 has seen a meaningful increase in FL applied specifically to wind energy, motivated by the data-silo problem identified above. Several published works are directly relevant to this thesis.

Taghizadeh et al. [40] propose DeepFedWT, a federated deep learning framework for wind turbine blade fault detection using SCADA data, demonstrating that FL can match centralised classification accuracy without data sharing. The framework uses standard FedAvg aggregation with no cryptographic privacy mechanism. Similarly, the authors of [41] propose a clustered FL framework for collaborative fault diagnosis across competing wind farms, grouping clients by data similarity before aggregation to address non-IID distributional shift. Again, no privacy mechanism beyond the basic FL protocol is applied.

The closest published work to this thesis in terms of privacy motivation is [42], which combines FL with differential privacy for wind turbine failure prediction across 31 turbines, using SCADA data and a deep learning architecture. DP is applied to gradient updates before aggregation, but secure aggregation is not used; the server receives and inspects individual per-client noisy gradients. Li et al. [43] apply FL with deep reinforcement learning to wind power forecasting, noting data privacy as a primary motivation but using no cryptographic privacy mechanism at the aggregation step.

Table 5 summarises the most directly relevant prior work, with the broader synthesis and the placement of this thesis within the landscape deferred to the gap analysis in Section 5.

Table 5: Selected prior work on federated learning applied to industrial and wind energy domains. H = horizontal FL. Privacy column indicates the mechanism applied to the aggregation step.

Paper (year)	Task	FL	Privacy mechanism	Key finding
Basnayake (2019) [11]	Sensor failure prediction	H	None	FL feasibility on wireless sensor data
Wang et al. (2019) [11]	Environment condition monitoring	H	None	FL accuracy comparable to centralised baseline
Hu et al. (2019) [11]	Multi-sensor time-series trend	H	None	FL within 5% of non-federated accuracy
Taghizadeh et al. (2022) [40]	Wind turbine blade fault detection	H	None	FL matches centralised classification on SCADA data
Clustered FL (2024) [41]	Wind farm fault diagnosis	H	None	Clustering mitigates non-IID degradation
DP wind FL (2024) [42]	Wind turbine failure prediction	H	DP (gradient noise)	DP applied but server inspects individual updates
Li et al. (2023) [43]	Wind power forecasting	H	None	FL with deep RL; privacy motivation but no SecAgg

4.4 System-Level Challenges of the Cross-Silo Regime

Overview

Three system-level challenges specific to the cross-silo, small- K setting distinguish the wind farm application from the deployments in which SecAgg was developed and evaluated.

Small client count

The SecAgg protocol [7, 13] was designed and evaluated for K in the hundreds to millions. Its privacy guarantee is conditional on a minimum number of surviving clients; if too many drop out before Round 3, the protocol aborts entirely rather than revealing individual updates. In the production deployment of Bonawitz et al. [7], 130% of the target number of devices is selected to offset dropout, a parameter calibrated empirically for the cross-device regime. In the wind farm setting, K may be as small as three (the number of farms in CARE) to a few dozen (turbines treated as independent clients), making this calibration more critical and the consequences of dropout more severe. Whether the protocol’s privacy and convergence properties degrade gracefully at small K has not, to the author’s knowledge, been characterised in the literature and is a primary motivation for RQ1.

Communication cost structure

In the cross-device regime, per-round communication cost is dominated by the sheer number of clients K , because each of the millions of mobile devices must exchange messages with the server in every round. In the cross-silo regime K collapses to a handful of clients, so the per-round cost is instead dominated by the size of each individual update: clients holding deep sequential models for SCADA regression may transmit update vectors with millions of parameters, and what was previously amortised over many small contributors is now concentrated in a few large ones. Bonawitz et al. [7] report that even in the cross-device setting, update sizes range from kilobytes to tens of megabytes per round; for the cross-silo case with comparable model architectures, the practical implication is that model compression and gradient quantisation become the principal levers for cost reduction, rather than client sub-sampling.

Data volume asymmetry

Wind farms differ in the number of turbines they operate, so client dataset sizes n_k may vary by an order of magnitude across participants (recall from Section 2.1 that n_k is the number of samples held by client k , and $n = \sum_k n_k$ is the total across all clients). This matters because FedAvg’s weighted aggregation,

$$w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_k^{t+1}, \quad (9)$$

gives each client k a weight proportional to its share of the total sample count. A client holding ten times the data of its peers therefore contributes ten times the influence on the global update, potentially biasing the model toward the operating conditions of the largest site. This asymmetry interacts with the non-IID distributional heterogeneity discussed in Section 4.1 and may need to be addressed in the aggregation design.

Several mitigations have been proposed in the FL literature. Uniform averaging discards n_k entirely and assigns each client equal weight, trading per-sample optimality for cross-client fairness. Capped weighting imposes an upper bound on the contribution of any single client, limiting the influence of large outliers. Clustered FL approaches such as that of [41] sidestep the issue by grouping clients with similar distributions and aggregating within groups before any cross-group combination. To the author’s knowledge, the comparative behaviour of these weighting strategies under cryptographic aggregation has not been systematically evaluated in a SCADA setting, and this is revisited in the gap analysis of Section 5.

5. Gap Analysis and Research Questions

5.1 Identified Gaps

The preceding review exposes five gaps in the existing literature. Each is stated as an explicit, evidence-supported claim.

Gap 1: No reference implementation integrates FL with SecAgg on industrial time-series regression

The FL literature contains mature theoretical treatments of SecAgg [13] and established applications of FL to sequential sensor prediction [11], but no published work combines both in a single system applied to real industrial data. Table 5 confirms this: every relevant prior work applies either FL without cryptographic privacy, or cryptographic protocols outside the federated aggregation step. Privacy guarantees well characterised in theory and on classification benchmarks may behave differently under a regression loss landscape and the feature structure of SCADA data.

Gap 2: SecAgg has not been evaluated on regression tasks

All substantive evaluations of SecAgg, including the 2017 protocol paper [13] and the 2019 at-scale deployment [7], use classification benchmarks. The privacy–utility trade-off under SecAgg for continuous-valued prediction, measured in terms of RMSE and MSE, has not been established. The masking dynamics and dropout behaviour of the protocol may differ across regression and classification loss landscapes in ways that are not yet understood.

Gap 3: SecAgg behaviour at small K is uncharacterised

The protocol’s dropout tolerance, privacy guarantees, and convergence interaction with FedAvg are all functions of K . The production deployment of Bonawitz et al. [7] operates at K in the hundreds to thousands, a regime well outside the cross-silo wind energy setting where K ranges from 3 (farm-level partition) to a few dozen (turbine-level partition). Whether the protocol’s properties degrade gracefully at small K is an open question not addressed in the existing literature.

Gap 4: The interaction between non-IID SCADA data and SecAgg is unexplored

It is established that non-IID data degrades FedAvg convergence [9, 11]. It is not known whether SecAgg masking amplifies, is orthogonal to, or partially compensates for this degradation in a regression setting. Since SCADA data is structurally non-IID across turbine sites due to differences in terrain, turbine model, and operating regime, this interaction is directly relevant to both the validity of the privacy guarantees and the utility of the learned model.

Gap 5: Wind energy FL research does not address cryptographic privacy at the aggregation step

Liu et al. [11] found that industrial manufacturing accounted for only $\approx 4\%$ of FL papers in their 2020 review, with no wind energy applications present. Since 2020, FL has been

applied to wind turbine fault detection [40,41], failure prediction [42], and power forecasting [43], confirming that the domain has attracted FL research. However, as established in Section 4.3, every published wind energy FL paper either applies no privacy mechanism or restricts itself to differential privacy. None applies SecAgg or any cryptographic mechanism to prevent the aggregator from inspecting per-client gradient updates. Guendouzi et al. [12] confirm that across the broader FL survey literature, this remains the case.

5.2 Research Questions

The five gaps above motivate the following two research questions, stated in the terms of the thesis proposal.

RQ1 (*Gaps 1, 2, 3*)

How can Secure Multiparty Computation primitives, particularly Secure Aggregation, be effectively integrated into a federated learning system to provide privacy guarantees during model update aggregation?

This question is addressed by the system design and prototype implementation: constructing a working FL system with SecAgg applied to a SCADA regression model, and characterising the protocol’s behaviour at the small client counts characteristic of cross-silo wind energy deployments.

RQ2 (*Gaps 2, 4, 5*)

To what extent do current secure aggregation protocols maintain individual client privacy when tested on real-world regression tasks using wind turbine SCADA telemetry?

This question is addressed by the evaluation phase: comparing prediction accuracy (RMSE, MSE) against the centralised baseline, assessing the privacy–utility trade-off under varying client counts and non-IID heterogeneity levels, and confirming that individual client gradients are not accessible to the aggregator under the implemented protocol.

6. Conclusion

6.1 Trends Observed

This review has surveyed the theoretical and empirical foundations of privacy-preserving federated learning, with particular attention to the cryptographic, statistical, and system-level considerations relevant to applying FL with SecAgg to wind turbine SCADA regression. Three convergent trends are evident across the literature.

Trend 1: Shift toward hybrid DP and SecAgg

The field is moving away from differential privacy as a standalone mechanism toward hybrid deployments that combine SecAgg with DP, providing both individual-gradient privacy and bounded aggregate information leakage. Bonawitz et al. [7] confirm this is now the state-of-the-practice in production FL systems, and McMahan et al. [35] demonstrate its viability for sequential models at scale. The complementary nature of the two mechanisms, established in Section 3.1, makes this convergence principled rather than merely pragmatic.

Trend 2: Growth of cross-silo FL in regulated sectors

Cross-silo FL is emerging as the practically dominant deployment mode in regulated industries, yet it remains far less studied than the cross-device setting. This gap is confirmed independently by Liu et al. [11] in 2020 and Guendouzi et al. [12] in 2023. The wind energy domain exemplifies this pattern: the past four years have seen meaningful FL adoption for fault detection and power forecasting, yet the privacy guarantees at the aggregation step remain unaddressed.

Trend 3: Maturation of public industrial benchmarks

Public industrial datasets such as CARE [36] are beginning to enable reproducible FL research in the energy sector, removing a barrier that previously limited empirical contributions. The availability of a well-annotated, multi-farm SCADA corpus with documented quality issues creates the conditions for rigorous evaluation of privacy-preserving FL that was not previously possible.

6.2 Conclusions

The review arrives at a consistent and well-supported conclusion. The theoretical foundations of SecAgg are mature [13, 24, 25]. The tooling is production-ready [27, 28]. The engineering challenges at scale are documented [7]. The threat landscape is well characterised [8, 14]. The application domain is well motivated [36], and post-2020 wind energy FL work [40–43] confirms that the field is active. What has not been published is the integration of these components in a working system applied to regression tasks on real industrial SCADA data at the small client counts characteristic of the cross-silo wind energy setting.

References

- [1] World Wind Energy Association, “Global wind statistics 2025: 1,347 GW installed, 11% of world electricity demand,” Bonn, 2026, accessed 18 May 2026. [Online]. Available: <https://wwindea.org/GlobalStatistics>
- [2] International Energy Agency, “Renewables 2025 — analysis and forecast to 2030,” Paris, 2025, accessed 18 May 2026. [Online]. Available: <https://www.iea.org/reports/renewables-2025>
- [3] European Wind Energy Association, “Operation and maintenance costs of wind-generated power (wind energy — the facts),” 2009, accessed 18 May 2026. [Online]. Available: <https://www.wind-energy-the-facts.org/operation-and-maintenance-costs-of-wind-generated-power.html>
- [4] Yenra, “AI predictive maintenance for wind turbines: 20 advances,” 2025, citing NREL analysis; accessed 18 May 2026. [Online]. Available: <https://yenra.com/ai20/predictive-maintenance-for-wind-turbines/>
- [5] European Parliament and Council of the European Union, “Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation),” Official Journal of the European Union, Tech. Rep. L 119, 2016, <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32016R0679>.
- [6] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282.
- [7] K. Bonawitz, H. Eichner, W. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kidon, J. Konečný, S. Mazzocchi, H. B. McMahan, T. Van Overveldt, D. Petrou, D. Ramage, and J. Roselander, “Towards federated learning at scale: A system design,” in *Proceedings of the 2nd Conference on Machine Learning and Systems (SysML)*, 2019, arXiv:1902.01046.
- [8] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, “Inverting gradients — How easy is it to break privacy in federated learning?” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 16 937–16 947.
- [9] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, “Advances and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021, arXiv:1912.04977 (2019).
- [10] C. Dwork and A. Roth, “The algorithmic foundations of differential privacy,” *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.

-
- [11] Y. Liu, L. Zhang, N. Ge, and G. Li, “A systematic literature review on federated learning: From a model quality perspective,” *arXiv preprint arXiv:2012.01973*, 2020.
 - [12] B. S. Guendouzi, S. Ouchani, H. El Assaad, and M. El Zaher, “A systematic review of federated learning: Challenges, aggregation methods, and development tools,” *Journal of Network and Computer Applications*, vol. 220, p. 103714, 2023.
 - [13] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, “Practical secure aggregation for privacy-preserving machine learning,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017, pp. 1175–1191.
 - [14] Y. Li, Z. Guo, N. Yang, H. Chen, D. Yuan, and W. Ding, “Threats and defenses in federated learning life cycle: A comprehensive survey and challenges,” *arXiv preprint arXiv:2407.06754*, 2024.
 - [15] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” in *Proceedings of Machine Learning and Systems (MLSys)*, 2020, arXiv:1812.06127 (2018).
 - [16] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, “SCAF-FOLD: Stochastic controlled averaging for federated learning,” in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 5132–5143.
 - [17] E. Fani, R. Camoriano, B. Caputo, and M. Ciccone, “Accelerating heterogeneous federated learning with closed-form classifiers,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, arXiv:2406.01116.
 - [18] F. Varno, M. Saghayi, L. Rafiee Sevyeri, S. Gupta, S. Matwin, and M. Havaei, “AdaBest: Minimizing client drift in federated learning via adaptive bias estimation,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022, pp. 710–726.
 - [19] Y. Li *et al.*, “Ultra-short-term wind power forecasting based on personalized robust federated learning with spatial collaboration,” *Energy*, 2023, scienceDirect, doi:10.1016/j.energy.2023.129714.
 - [20] Anonymous, “Privacy-preserving probabilistic wind power forecasting: An adaptive federated approach,” *Applied Energy*, 2025, scienceDirect, doi:10.1016/j.apenergy.2025.125000.
 - [21] H. Wang, M. Yurochkin, Y. Sun, D. Papailiopoulos, and Y. Khazaeni, “Federated learning with matched averaging,” in *International Conference on Learning Representations (ICLR)*, 2020, arXiv:2002.06440.
 - [22] M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary, “Federated learning with personalization layers,” *arXiv preprint arXiv:1912.00818*, 2019.

-
- [23] Z. Liu, S. Chen, J. Ye, J. Fan, H. Li, and X. Li, “DHSA: Efficient doubly homomorphic secure aggregation for cross-silo federated learning,” *The Journal of Supercomputing*, vol. 79, no. 3, pp. 2819–2849, 2023.
 - [24] A. Shamir, “How to share a secret,” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.
 - [25] M. Ben-Or, S. Goldwasser, and A. Wigderson, “Completeness theorems for non-cryptographic fault-tolerant distributed computation,” in *Proceedings of the 20th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 1988, pp. 1–10.
 - [26] J. H. Bell, K. A. Bonawitz, A. Gascón, T. Lepoint, and M. Raykova, “Secure single-server aggregation with (poly)logarithmic overhead,” in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2020, pp. 1253–1269.
 - [27] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, H. L. Kwing, T. Parcollet, P. P. B. de Gusmão, and N. D. Lane, “Flower: A friendly federated learning research framework,” *arXiv preprint arXiv:2007.14390*, 2020.
 - [28] Flower Labs, “SecAgg+: Practical secure aggregation in Flower,” <https://flower.ai/docs/examples/flower-secure-aggregation.html>, 2023.
 - [29] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, “Membership inference attacks against machine learning models,” in *Proceedings of the 2017 IEEE Symposium on Security and Privacy (S&P)*, 2017, pp. 3–18.
 - [30] Y. Qu, S. R. Pokhrel, S. Garg, L. Gao, and Y. Xiang, “A blockchained federated learning framework for cognitive computing in industry 4.0 networks,” *IEEE Transactions on Industrial Informatics*, vol. 17, no. 4, pp. 2964–2973, 2021.
 - [31] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, “Deep learning with differential privacy,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2016, pp. 308–318.
 - [32] I. Mironov, “Rényi differential privacy,” *IEEE Computer Security Foundations Symposium (CSF)*, 2017, arXiv:1702.07476.
 - [33] J. Jiang *et al.*, “Pack: Towards communication-efficient homomorphic encryption in federated learning,” in *Proceedings of the 2024 ACM Symposium on Cloud Computing (SoCC)*, 2024.
 - [34] P. Mohassel and Y. Zhang, “SecureML: A system for scalable privacy-preserving machine learning,” in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2017, pp. 19–38.
 - [35] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, “Learning differentially private recurrent language models,” in *International Conference on Learning Representations (ICLR)*, 2018, arXiv:1710.06963.

-
- [36] J. Gück *et al.*, “CARE: A wind turbine SCADA dataset for fault detection covering coverage, accuracy, reliability and earliness,” *[TODO: confirm venue]*, 2024, [TODO: add DOI/URL after checking paper PDF].
- [37] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021, <https://otexts.com/fpp3/>.
- [38] ACM SIGKDD, “Applied data science (ADS) track call for papers,” ACM Knowledge Discovery and Data Mining conference, 2024, accessed 18 May 2026. [Online]. Available: <https://kdd2024.kdd.org/applied-data-science-ads-track-call-for-papers/>
- [39] J. Zhou, X. Lu, Y. Xiao *et al.*, “SDWPF: A dataset for spatial dynamic wind power forecasting over a large turbine array,” *Scientific Data*, vol. 11, 2024, kDD Cup 2022 dataset.
- [40] A. Taghizadeh *et al.*, “DeepFedWT: A federated deep learning framework for fault detection of wind turbines,” *Measurement*, 2022, scienceDirect, doi:10.1016/j.measurement.2022.111529.
- [41] and others, “A clustered federated learning framework for collaborative fault diagnosis of wind turbines,” *Applied Energy*, 2024, scienceDirect, doi:10.1016/j.apenergy.2024.124190.
- [42] —, “A differential privacy-preserving federated learning scheme for predictive maintenance of wind turbines,” *Journal of Mechanical Science and Technology*, 2024, springer, doi:10.1007/s12206-024-0616-9.
- [43] Y. Li *et al.*, “Wind power forecasting considering data privacy protection: A federated deep reinforcement learning approach,” *Applied Energy*, vol. 329, 2023, art. no. 120291.